Chapter 13 — Modern CSS and Architecture

Until about 2022, CSS for a big site meant tools: Sass (a language compiled into plain CSS) for nesting and variables, strict naming conventions so styles didn't collide, and JavaScript for things like "style the parent when a child is checked". Browsers have caught up: nesting, `:has()`, cascade layers, `@scope`, logical properties and `light-dark()` are built in, and they change how you organise stylesheets. This chapter covers each feature, then the question every team faces — **how do you structure CSS so it stays maintainable as a project grows?** — comparing BEM, utility-first (Tailwind CSS), CSS Modules and CSS-in-JS, plus Sass, resets and a complete theming system.

In this chapter: 13.1 Native nesting · 13.2 `:is()` and `:where()` · 13.3 `:has()` · 13.4 Cascade layers · 13.5 `@scope` · 13.6 Logical properties · 13.7 Why CSS needs an architecture; BEM · 13.8 Utility-first and Tailwind CSS v4 · 13.9 CSS Modules · 13.10 CSS-in-JS · 13.11 Choosing an approach · 13.12 Sass and PostCSS · 13.13 Resets and normalize · 13.14 Theming and dark mode.

How the numbers and figures were made. Every example is a page in `code/`. Each `NN_*.mjs` script opens its page in headless Chrome 154 via `code/measure.mjs` (Chapter 10's helper) and prints what the browser computed — `getComputedStyle()`, bounding boxes and the CSSOM (the browser's object model of the stylesheet). The Tailwind example is a real Tailwind CSS v4.3 build; the CSS Modules example is compiled by Lightning CSS (13.12), the compiler many **bundlers** (build tools that combine a project's source files into a few optimised ones) use. `code/make_figures.mjs` takes the screenshots. Support data is from the **web-features** dataset (behind MDN's and caniuse's "Baseline" badges — §1.1: *newly available* = in all core browsers, *widely available* = 30 months later), checked September 2026. For older devices, check your analytics or provide a fallback.

13.1 Native CSS nesting

What it is

Nesting writes a rule inside another, so the inner selector is relative to the outer. Sass has had it since 2006; browsers do it natively: **Baseline newly available since December 2023** (Chrome 120, Firefox 117, Safari 17.2), **widely available since June 2026**.

Why it matters

Without nesting you repeat the parent selector for every child, hover state and media query, and related rules drift apart. Nesting keeps a component in one block — with no **build step** (a program transforming sources before serving), and DevTools shows the rule as written.

How it works

1. Inside a style rule you can write declarations **and** further rules.
2. A nested selector with no `&` is a **descendant** of the parent: `.title {}` inside `.card {}` = `.card .title` (write `> .title` for a child).
3. `&` (the **nesting selector**) stands for the parent's matched elements; its position sets the relationship: `&:hover` (same element), `& > p` (child), `.dark-page &` (parent *inside* `.dark-page`), `& + &` (an item right after another).
4. A nested selector may start with a **combinator** (space, `>`, `+`, `~`): `> p` means `& > p`.
5. Conditional at-rules (`@media`, `@supports`, `@container`, `@layer`, `@scope`) nest too; their declarations apply to the parent selector.
6. `&` behaves like `:is(<parent selector list>)` — which matters for specificity (§6.4).

`code/01_nesting.html`:

```
.card {
  padding: 16px;
  border: 2px solid #cbd5e1;

  .title { font-size: 20px; margin: 0; }          /* = .card .title (descendant)        */
  > p { margin: 4px 0 0; color: #334155; }         /* = .card > p (relative selector)    */
  &:hover { border-color: #2563eb; }               /* = .card:hover (no space)           */
  &.featured { border-color: #f59e0b; }            /* = .card.featured (same element)    */
  .dark-page & { background: #0f172a; }            /* = .dark-page .card (& at the end)  */
  @media (width >= 600px) { padding: 24px; }       /* nested at-rule: applies to .card   */
  &__body { color: red; }                          /* NOT Sass: no string joining        */
}

/* & behaves like :is() -> the parent list's HIGHEST specificity is used */
#main, .sidebar {
  .link { color: crimson; }                         /* = :is(#main, .sidebar) .link  (1,1,0) */
}
.sidebar .link.plain { color: green; }              /* (0,3,0) -- still loses!              */
```

code/01_nesting.mjs walks the CSSOM and reads computed styles at two viewport widths:

```
CSSOM tree (selectorText of each rule):
body
.card
  & .title
  & > p
  &:hover
  &.featured
  .dark-page &
  @media (width >= 600px)
    CSSNestedDeclarations
#main, .sidebar
  & .link
.sidebar .link.plain

Was the "&__body" rule kept? false

viewport 500px: .card padding 16px, border rgb(245, 158, 11), background rgb(15, 23, 42)
  .title font-size 20px   .card__body color rgb(0, 0, 0) (not red)   .sidebar .link color rgb(220, 20, 60)

viewport 700px: .card padding 24px, border rgb(245, 158, 11), background rgb(15, 23, 42)
  .title font-size 20px   .card__body color rgb(0, 0, 0) (not red)   .sidebar .link color rgb(220, 20, 60)
```

- The browser stores `.title` as `& .title` and `> p` as `& > p` — an implicit `&` and descendant space.
- The nested `@media` holds a `CSSNestedDeclarations` object (bare `padding: 24px` treated as `& { … }`): 16px at 500px, 24px at 700px.
- **`&__body` was thrown away.** Sass glues text into `.card__body`; native CSS works on selectors, not strings, so Chrome discards the rule. The #1 surprise when moving BEM-style (13.7) Sass to native nesting (same for `&-large`).
- The sidebar link is **crimson** although `.sidebar .link.plain` (0,3,0) looks stronger: the nested rule is `:is(#main, .sidebar) .link`, and `:is()` takes its most specific argument — `#main` — giving (1,1,0) even though the element matched via `.sidebar`.

Compiling nesting away for old browsers

To support pre-late-2023 browsers, a build tool can **lower** (flatten) nesting. code/01_nesting_lower.mjs asks Lightning CSS to target Chrome 100:

```css
.card {
  padding: 16px;
}

.card .title {
  font-size: 20px;
}
/* ... .card:hover, .dark-page .card ... */

@media (min-width: 600px) {
  .card {
    padding: 24px;
  }
}

:is(#main, .sidebar) .link {
  color: #dc143c;
}
```

It writes `:is(#main, .sidebar) .link` — the browser's meaning (Sass writes `#main .link, .sidebar .link`, two selectors with *different* specificities) — and rewrote the range media query `(width >= 600px)` to `min-width`.

Common mistakes

- Porting Sass `&__title` or `&-large`.
- **Nesting too deep** — `.page .main .card .body .title a` nested is as fragile as flat. Keep to 2–3 levels: states, owned children, media/container queries.
- Forgetting a nested rule inherits the `:is()` specificity of the *whole* parent list.

In the interview

30-second answer: "CSS supports nesting natively — Baseline since late 2023. A nested selector is relative to its parent; `&` represents the parent and behaves like `:is()` of the parent list, so it takes the highest specificity in that list. Unlike Sass it doesn't concatenate strings, so `&__title` doesn't work. I use it for states, owned children and nested media queries, kept shallow; a build tool is needed only to lower it for pre-2023 browsers."

13.2 `:is()` and `:where()`

What it is

Both take a **selector list** and match an element matching any entry. They differ only in **specificity**: `:is()` counts as its **most specific argument** (`:is(p, .x, #y)` = (1,0,0) whichever matched); `:where()` always counts **zero** (`:where(#a) .b` = (0,1,0)). Baseline widely available since 2021. Chapter 6 covers the scoring; here they're architecture tools.

Why it matters

1. **Shorter selectors**: `:is(article, section, aside) :is(h1, h2, h3)` replaces nine.
2. **Forgiving parsing**: one invalid selector in a normal list drops the **whole rule**; inside `:is()` / `:where()` bad entries are just ignored — safe for new pseudo-classes.
3. **Specificity control**: `:where()` for defaults anything can override (resets, libraries); `:is()` to raise specificity.

Proof

`code/02_is_where.html` :

```
:is(article, section, aside) :is(h1, h2, h3) { color: rebeccapurple; }

:is(.promo, :unknown-thing) { font-weight: 700; }           /* forgiving   */
.promo, :unknown-thing { text-decoration: underline; }      /* NOT forgiving */

:where(.content) a  { color: gray; }       /* (0,0,1)                     */
a                   { color: teal; }       /* (0,0,1), later -> wins      */
:is(#nav, .content) b { color: gray; }     /* (1,0,1)                     */
.content b.hot      { color: teal; }       /* (0,2,1) -> loses            */
```

code/02_is_where.mjs reads computed styles and scores selectors with Chapter 6's calculator:

```
h2 inside article color      rgb(102, 51, 153) (rebeccapurple = rgb(102, 51, 153))
.promo font-weight           700   <- forgiving :is() kept the rule
.promo text-decoration-line  none   <- plain list with one bad selector was dropped
:where(.content) a vs a      rgb(0, 128, 128) (teal = rgb(0, 128, 128))
:is(#nav,.content) b vs .content b.hot rgb(128, 128, 128) (gray = rgb(128, 128, 128))

Rules the browser kept: body  |  :is(article, section, aside) :is(h1, h2, h3)  |  :is(.promo)  |  :where(.content) a  |  a  |  :is(#nav, .content) b  |  .content b.hot

Specificity (A,B,C):
  :is(article, section, aside) :is(h1, h2, h3)   (0,0,2)
  :where(.content) a                             (0,0,1)
  a                                              (0,0,1)
  :is(#nav, .content) b                          (1,0,1)
  .content b.hot                                 (0,2,1)
```

Chrome kept `:is(.promo)` (silently dropping `:unknown-thing`) while `.promo, :unknown-thing` is **missing**; and `:is(#nav, .content) b` wins with an ID's weight though no `#nav` exists.

Where each is used

Use	Pick	Example
Reset / library defaults that must be easy to override	`:where()`	`:where(ul, ol)[role="list"] { list-style: none }`
Shortening a long list without changing priorities much	`:is()`	`:is(h1,h2,h3):hover`
Using a new pseudo-class safely next to old ones	either (forgiving)	`:is(:focus-visible, .focus)`
Deliberately raising a selector's weight	`:is()` with an ID or class	`.btn:is(#_, .btn)` (a hack — prefer layers, 13.4)

Common mistakes

- `:is()` in a library's defaults: an `#app` argument makes everything inside hard to override. Use `:where()`.
- Pseudo-elements inside: `:is(::before)` never matches — not allowed in these lists.

13.3 `:has()` — the relational ("parent") selector

What it is

`A:has(B)` matches **A** if **B**, evaluated *relative to A*, matches something. With a descendant (`:has(img)`) it's the long-awaited **parent selector**; with `+` / `~` (`h3:has(+ .subtitle)` , `h2:has(+ p)`) a **previous-sibling** selector. Baseline newly available since **December 2023** (Firefox 121 last), widely available since June 2026. Basic syntax: §6.3.

The problem it solves

Selectors used to look only **down and forward**. "Grid layout if the card has an image", "highlight the label whose radio is checked", "lock scrolling while a dialog is open" needed JS adding classes; `:has()` does them in CSS, always in sync.

How it works

1. The argument is a **relative selector list**; each item implicitly starts at the `:has()` element (default combinator: descendant).
2. Specificity = most specific argument, like `:is()` (`.card:has(> img, #hero)` = (1,1,0)).
3. Re-evaluated whenever the DOM or state changes (checking a box, typing, opening a dialog).
4. Restrictions: no `:has()` inside `:has()`; no pseudo-element arguments.

`code/03_has.html` shows six patterns:

```
.card:has(img)                { display: grid; grid-template-columns: 64px 1fr; }  /* parent       */
.field:has(input:invalid)     { border-left-color: #dc2626; background: #fef2f2; } /* form state   */
h3:has(+ .subtitle)           { margin-bottom: 0; color: #7c3aed; }                /* prev sibling */
ul:has(> li:nth-child(4))     { columns: 2; }                                      /* quantity     */
label.opt:has(input:checked)  { background: #dbeafe; font-weight: 600; }           /* checked      */
html:has(dialog[open])        { overflow: hidden; }                                /* page-level   */
```



Figure 13.1 — Every box's styling comes from a `:has()` rule; no JavaScript adds classes.

`code/03_has.mjs` reads styles, changes state like a user (click "Yearly", fix the email, add a 4th item, open the dialog), and reads again:

```
.card with img display=grid, without img display=block
invalid field border=rgb(220, 38, 38), valid field border=rgb(203, 213, 225)
h3 followed by .subtitle margin-bottom=0px, lone h3 margin-bottom=8px
3-item list columns=auto, 5-item list columns=2
Monthly (checked) weight=600, Yearly weight=400
<html> overflow with dialog closed: visible

after: click "Yearly", fix the email, add a 4th <li>, showModal()
Monthly weight=400, Yearly weight=600
first field border=rgb(203, 213, 225)
first list columns=2
<html> overflow with dialog open: hidden

a:has(:has(b))  -> SyntaxError
p:has(::before)  -> SyntaxError
li:has(+ li)  -> valid
```

Every style followed the state change instantly; the two forbidden forms are rejected by the selector parser. (Exactly 3 items: `ul:has(> li:nth-child(3)):not(:has(> li:nth-child(4)))`.)

Trade-offs and performance

`:has()` makes the browser look "up" the tree on changes — more work, but engines optimise common cases. Keep it cheap: **anchor it** (`.card:has(img)`, not bare `:has(img)` or `body:has(.x .y .z)`), and avoid `*:has(...)` on huge, constantly changing DOMs.

When not to use it

- For *behaviour* (disabling a button, moving focus) — JavaScript's job; `:has()` only changes appearance.
- Error styles at page load: `:invalid` matches before typing; use `.field:has(:user-invalid)` (Chapter 5).

Real-world examples

Selected table rows (`tr:has(input:checked)`), error-highlighting form groups, quantity queries (styles by child count), modal scroll locking, dimming siblings of a hovered item: `.list:has(.item:hover) .item:not(:hover) { opacity: .6 }`.

In the interview

30-second answer: "`:has()` matches an element if the relative selector inside matches — `.card:has(img)` styles cards containing an image. It's a parent selector and, with `+` / `~`, a previous-sibling selector, and it reacts to state like `:checked` (`label:has(input:checked)`) or `:invalid`. Baseline since late 2023. I anchor it for performance and never use it for behaviour."

13.4 Cascade layers (`@layer`)

What it is

A **cascade layer** is a named bucket of rules. Declare the bucket order once, and a rule in a later layer beats one in an earlier layer **however specific either selector is** (like floors of a building). Baseline since March 2022, widely available since September 2024. §6.5 placed layers in the cascade; this section uses them to structure a project.

The problem it solves

Specificity wars: a library ships `#app .btn.btn-primary`; you write something more specific or `!important`; the next person escalates. Layers say, once, "vendor CSS is below my components, utilities above both".

How it works

1. `@layer reset, vendor, base, components, utilities;` — **first = weakest**. Put this statement first so order doesn't depend on file load order (a layer's position is fixed by its first appearance).
2. Add rules with `@layer name { … }` (blocks merge), or import a file into a layer: `@import url(x.css) layer(vendor);` (must precede all rules except `@charset` and `@layer` statements, or it's ignored).
3. The cascade compares **origin & importance → context → style attribute → layer → specificity → order** — layer **before** specificity.
4. **Unlayered** styles form an implicit final layer: they beat every named layer.
5. `!important` **reverses the order**: an earlier layer's `!important` beats a later one's, and layered `!important` beats unlayered `!important` — letting a reset lock something down.
6. `revert-layer` rolls a property back to what the earlier layers (or earlier origins) produce.
7. Layers nest: `@layer components { @layer dialog { … } }` → `components.dialog`.

code/04_layers.html (with code/04_vendor.css as a library):

```
@layer reset, vendor, base, components, utilities;
@import url("04_vendor.css") layer(vendor);     /* #app .btn.btn-primary { background: #6b7280; ... } */

@layer components { .btn { background: #2563eb; border-radius: 8px; }  .card a { color: #7c3aed; } }
@layer utilities  { .text-red { color: #dc2626; }  .no-radius { border-radius: 0 !important; } }
@layer reset      { .keep-radius { border-radius: 999px !important; } }
p { color: #065f46; }                                        /* unlayered */
@layer utilities  { .plain-link { color: revert-layer; } }
@layer components { @layer dialog { .dlg { color: #b45309; } } }
```

code/04_layers.mjs lists the layers from the CSSOM and reads each fight's winner:

```
statement: reset, vendor, base, components, utilities
import:    04_vendor.css layer(vendor)
block:     reset
block:     base
block:     components
block:     utilities
block:     components.dialog

#app .btn.btn-primary (vendor, 1,2,0) vs .btn (components, 0,1,0):
   background rgb(37, 99, 235) (#2563eb = rgb(37, 99, 235)), radius 8px, padding-left 20px (vendor's, nobody
overrode it)
.card a (components)                    color rgb(124, 58, 237)
.text-red (utilities) vs .card a        color rgb(220, 38, 38)
.plain-link { color: revert-layer }     color rgb(124, 58, 237)  <- back to the components value
unlayered p vs base body                color rgb(6, 95, 70)
!important: utilities 0 vs reset 999px   radius 999px  <- earlier layer wins for !important
nested layer components.dialog         color rgb(180, 83, 9)
```

A one-class selector in `components` beat an ID selector in `vendor`; the vendor's padding survived because nothing higher set padding (layers settle *conflicts*, they don't delete rules); `revert-layer` fell back to the components colour; for `!important`, **reset** won.

A layer order for real projects

```
@layer reset, tokens, vendor, base, layout, components, utilities, overrides;
```

Layer	Holds	Why here
`reset`	the reset (13.13)	weakest: everything may override it
`tokens`	custom properties (`--brand` , 13.14)	declarations only, rarely conflicts
`vendor`	third-party CSS via `@import … layer(vendor)`	your base styles should beat it
`base`	element defaults (`body` , `a` , `h1`)	
`layout`	page grids, containers	
`components`	cards, buttons, dialogs	
`utilities`	single-purpose classes (`.visually-hidden` , `.text-center`)	must beat components
`overrides`	temporary fixes	explicit, easy to find and delete

Tailwind v4 emits `@layer theme, base, components, utilities;` (13.8), slotting neatly into this scheme.

Common mistakes

- **Your CSS unlayered while the framework is layered**: unlayered beats all layers — maybe intended, or maybe your stray rules become impossible to override with utilities.
- **Order by accident** — no up-front statement, so an early file creates layers in the wrong order.
- Expecting a later layer's `!important` to win — it's the reverse.

In the interview

> **30-second answer:** "Cascade layers order groups of rules explicitly. The cascade compares layers before specificity, so a simple class in a later layer beats an ID selector in an earlier one. I declare the order once — reset, vendor, base, components, utilities — and import third-party CSS into a low layer, which is how I override a library without `!important` . Unlayered styles beat all layers, and `!important` inverts the order."

13.5 `@scope`

What it is

`@scope (<root>) to (<limit>) { … }` applies rules only to elements **inside** a scope root and, optionally, **not inside** a lower boundary — a **donut scope** (root = ring, limit = hole). Inside, `:scope` is the root element itself.

Status (September 2026): Baseline newly available since March 2026. Chromium shipped it first (Chrome 118, 2023); web-features counts it complete from Firefox 146 and Safari 26.4. Older browsers ignore the whole block, so give unscoped fallbacks for essentials.

The problem it solves

- **Leaking into content**: `.card img { border: … }` also hits images in user-generated content inside the card. BEM avoids this with long class names; `@scope … to (.content)` does it with a boundary.
- **Proximity**: with nested themes (`.light` in `.dark` in `.light`), `.dark a` and `.light a` tie on specificity, so the later one wins — wrong for the innermost box. In `@scope` , when specificity ties, the rule whose root is **nearest** (fewest steps up) wins, regardless of source order.

How it works

1. Match the scope root(s) (`.card`).
2. Match the limit(s) (`.content`) inside each root; everything at or below a limit is excluded.
3. Apply the rules in between; bare selectors like `img` act as descendants of `:scope` .

4. Specificity: the root selector adds **nothing** (unlike nesting — `@scope (#main)` adds nothing); `:scope` adds (0,1,0).
5. **Scoping proximity** is a new cascade step *after* specificity, *before* order of appearance.
6. `@scope { … }` **without a prelude** (no root/limit) inside a `<style>` scopes to that `<style>`'s parent — component-local styles in HTML.

`code/05_scope.html` :

```
@scope (.card) to (.content) {
  :scope { background: #f8fafc; }          /* the .card itself                */
  img    { border: 3px solid #2563eb; }    /* card's images, not content's    */
  p      { font-weight: 600; color: #1d4ed8; }
}
@scope (.dark)  { a { color: #fbbf24; } :scope { background: #1e293b; color: #e2e8f0; } }
@scope (.light) { a { color: #1d4ed8; } :scope { background: #fff; color: #0f172a; } }
```

```
<div id="inline">
  <style> @scope { :scope { outline: 2px dashed #16a34a; } b { color: #16a34a; } } </style>
  <b>styled by an inline, prelude-less @scope</b>
</div>
<b>same &lt;b&gt;, outside that div</b>
```

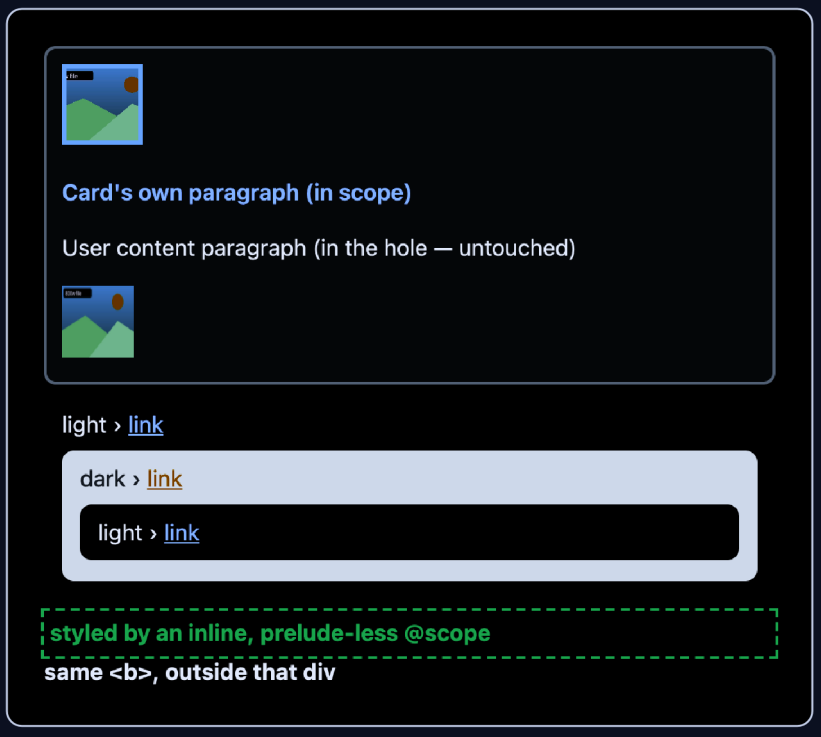


Figure 13.2 — Donut scope (top), proximity with nested themes (middle), prelude-less inline scope (bottom).

`code/05_scope.mjs` :

```
CSSScopeRule supported: true
start=.card end=.content
start=.dark end=null
start=.light end=null

.card background rgb(248, 250, 252)
card's own img border 3px   img inside .content border 0px
card's own p  rgb(29, 78, 216) 600
p in .content rgb(15, 23, 42) 400

links: light rgb(29, 78, 216)   dark rgb(251, 191, 36)   light-inside-dark rgb(29, 78, 216)
(amber = rgb(251, 191, 36), blue = rgb(29, 78, 216))

inline @scope: <b> inside rgb(22, 163, 74), <b> outside rgb(15, 23, 42)
```

The hole works (content's `img` and `p` untouched), and the innermost **light** link is blue because the nearest root decided. Without `@scope`, the later `.light a` would also colour the *dark* box's link blue.

`@scope` vs the alternatives

Need	Tool
Keep a component's styles from matching outside it	`@scope (.card)`, BEM names, CSS Modules
Stop styles at a nested region (slots, user content)	`@scope … to (…)` — only this does it natively
Nearest theme wins	`@scope` proximity (or custom properties, which inherit from the nearest ancestor anyway — 13.14)
Full isolation both ways (page CSS can't get in)	Shadow DOM (a hidden, separately styled DOM tree attached to an element; used by web components)

Common mistakes

- Expecting `@scope` to block **inheritance**: inherited properties (colour, font) still flow into the hole; only *selector matching* stops.
- Thinking the root adds specificity.

13.6 Logical properties

What it is

Logical properties describe boxes by the **flow of text**, not physical sides. The **inline** axis is the direction text runs in a line (left→right in English, right→left in Arabic, top→bottom in vertical Japanese); the **block** axis is how lines stack; `start` / `end` are each axis's beginning and end. So `margin-inline-start` is the margin **before the text starts** — left in English, right in Arabic. Baseline widely available since 2021.

Why it matters

Arabic, Hebrew, Persian and Urdu display right-to-left (`dir="rtl"`; RTL vs LTR). With physical properties (`padding-left`, `border-left`, `left: 0`) every side needs flipping in an RTL stylesheet; with logical properties **one stylesheet works for both**, and for vertical `writing-mode`s.

The mapping

Physical (horizontal LTR)	Logical
`width` / `height`	`inline-size` / `block-size` (also `min-` / `max-`)
`margin-left` / `margin-right`	`margin-inline-start` / `margin-inline-end`
`margin-top` / `margin-bottom`	`margin-block-start` / `margin-block-end`
`margin-left` + `margin-right`	`margin-inline: a b` (shorthand; `padding-inline: 1rem` = left + right padding)
`padding-*`, `border-*`	`padding-inline`, `padding-block-start`, `border-inline-start`, …
`top` / `right` / `bottom` / `left`	`inset-block-start` / `inset-inline-end` / `inset-block-end` / `inset-inline-start`
`border-top-left-radius`	`border-start-start-radius` (block-start, inline-start)
`text-align: left` / `right`	`text-align: start` / `end`
`float: left`	`float: inline-start`
`overflow-x` / `overflow-y`	`overflow-inline` / `overflow-block` (newer — check support)

Units too: `vi` / `vb` (viewport) and `cqi` / `cqb` (containers, §10.6).

Proof

`code/06_logical.html` applies one rule set to `dir="ltr"`, `dir="rtl"` and `writing-mode: vertical-rl` containers:

```
.note {
  inline-size: 220px;
  padding-inline-start: 36px;  padding-inline-end: 48px;  padding-block: 10px;
  border-inline-start: 6px solid #2563eb;
  margin-block-end: 12px;
  text-align: start;
}
.note .badge { position: absolute; inset-inline-end: 6px; inset-block-start: 6px; }
```

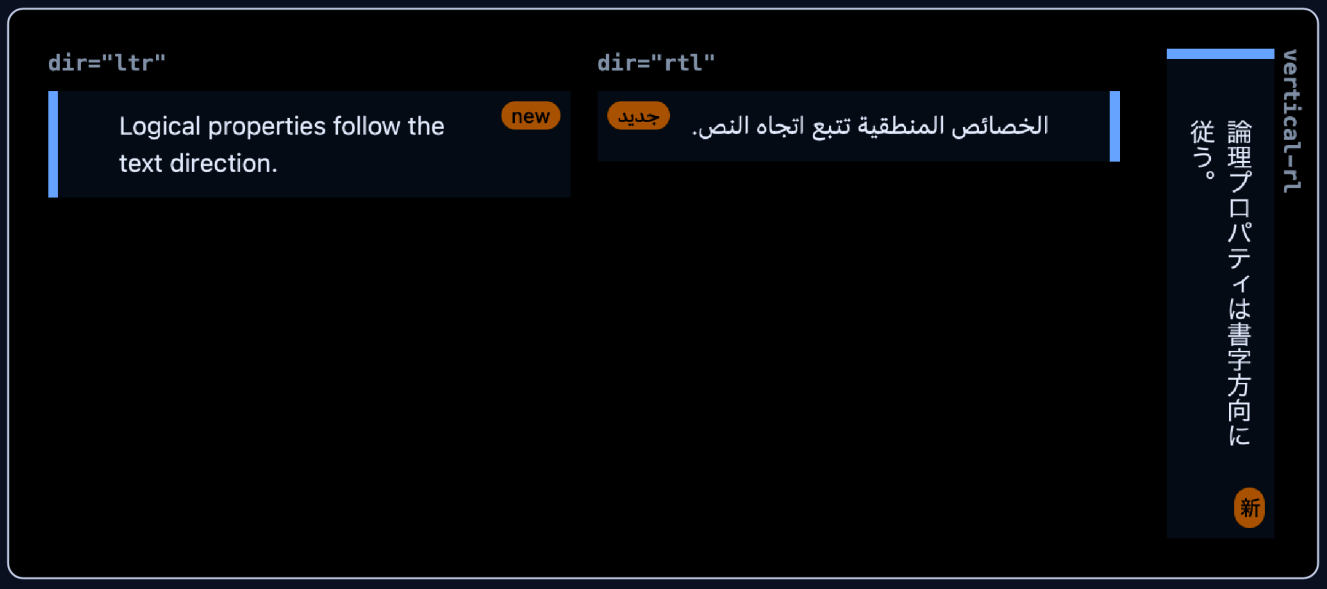


Figure 13.3 — Same CSS, three writing directions; the accent bar is always at the "start" of the text.

`code/06_logical.mjs` measures which **physical** side got each value:

```
box  size(px)   36px padding  48px padding  6px border  12px margin  badge corner
ltr  310x64     Left          Right         Left        Bottom       right/top
rtl  310x42     Right         Left          Right       Bottom       left/top
vrl  64x290     Top           Bottom        Top         Left         right/bottom
```

In vertical-rl, lines run top→bottom (inline axis vertical) and stack right→left, so inline-start = **top**, inline-end = **bottom**, block-end = **left**. LTR/RTL boxes are 310px **wide** (220 + 36 + 48 + 6); the vertical box (`inline-size: 200px`) is 290px **tall** (200 + 36 + 48 + 6). In RTL, `border-inline-end` draws on the left; in horizontal writing `block-size` is height.

When not to use it

- Genuinely physical things: a downward shadow, a fixed bottom-right "back to top" button, an arrow that points right because the image does.
- Don't mix `margin-left` and `margin-inline-start` on one element — the same side in LTR, so they fight.

Common mistakes

- `dir="rtl"` on `<html>` but hard-coded `left` / `right` in components.
- Directional icons (→, "next") must mirror in RTL: `:dir(rtl) .icon-next { scale: -1 1; }` (`:dir()` Baseline since 2023).

13.7 Why CSS needs an architecture — and BEM

The problems

CSS was designed for documents; at application scale its core ideas hurt:

Problem	What happens	Symptom
Global namespace (one shared pool of names)	Every selector can match anything on any page	Changing `.title` breaks a page you've never seen
Specificity & order dependence	Winners depend on selector weight and file order	Escalation: `!important` , `#id` hacks
No dead-code detection	You can't tell if a rule is still used	Stylesheets only grow ("append-only CSS")
Implicit coupling	Styles depend on DOM structure (`.nav ul li a`)	Moving markup breaks styles

Architectures are answers in two families: **conventions** humans follow (BEM, OOCSS, SMACSS, ITCSS, CUBE CSS) and **tooling** that enforces them (CSS Modules, CSS-in-JS, utility generators). Native features (layers, `@scope` , nesting) help both.

BEM — Block, Element, Modifier

BEM (Yandex, ~2009) gives every styled thing a unique, flat class:

- **Block** — a standalone component: `.card`
- **Element** — a part meaningful only inside its block: `.card__title` (double underscore)
- **Modifier** — a variant or state: `.card--featured` , `.btn__icon--large` (double hyphen)

code/07_bem.html :

```
.card { width: 170px; padding: 14px; border: 1px solid #cbd5e1; border-radius: 12px; }
.card__title  { margin: 0; font-size: 16px; }
.card__price  { margin: 4px 0 12px; color: #475569; }
.card__button { width: 100%; background: #2563eb; color: #fff; /* … */ }
.card--featured { border: 2px solid #f59e0b; background: #fffbeb; }
.card--featured .card__button { background: #d97706; }
```

```
<article class="card card--featured">
  <h3 class="card__title">Rain jacket</h3>
  <p class="card__price">$129</p>
  <button class="card__button">Add to cart</button>
</article>
```

Why it works: nearly every selector is one class → (0,1,0) everywhere, so weight and order stop mattering; names say where things belong; no dependence on tags or nesting depth (`.card h3` would couple to markup and raise specificity). **Rules:** no element-of-element (`card__body__title` → `card__title`); modifiers never alone (`class="card--featured"` without `card` is a bug); avoid styling by tag inside blocks.

Other conventions in one line each

- **OOCSS** (Object-Oriented CSS, Nicole Sullivan, 2008): separate *structure* from *skin* and *container* from *content* — reusable "objects" like `.media`.
- **SMACSS**: rules sorted into Base, Layout, Module, State (`.is-open`), Theme.
- **ITCSS** (Inverted Triangle CSS): files ordered generic/low-specificity → explicit/high (settings → tools → generic → elements → objects → components → utilities) — cascade layers (13.4) now express this natively.
- **CUBE CSS**: Composition, Utility, Block, Exception — lean on the cascade, add blocks only where needed, exceptions via data attributes.

Trade-offs

BEM needs discipline and code review (nothing enforces names), names get long, and there's still one global namespace — two teams can both invent `.card`. Tooling (13.9, 13.10) and `@scope` solve that last part.

13.8 Utility-first CSS and Tailwind CSS v4

What it is

Utility-first CSS provides many tiny single-purpose classes — `flex` , `p-4` , `text-slate-600` , `rounded-lg` — composed **in the HTML** instead of writing CSS per component. **Tailwind CSS** is the dominant framework; **v4** (January 2025; 4.3.3 here) moved configuration into CSS and builds on cascade layers, `@property` (registers a typed custom property the browser can animate and validate), `color-mix()` and custom properties for every **design token** (a named design decision — brand colour, spacing step, radius — defined once, reused everywhere).

Why teams choose it

- **CSS stops growing**: new components reuse utilities, so size tracks *distinct* utilities, not components.
- **No naming**, no file switching; deleting HTML deletes its styles (no dead CSS).
- **Constraints**: utilities come from a design scale, keeping UIs consistent.
- **Generated on demand**: the build emits only classes actually used.

How a v4 build works

1. Your CSS entry imports Tailwind and optionally defines tokens in `@theme` .
2. The build scans sources for strings that look like class names (it doesn't run your code — names must appear **complete**).
3. Each recognised class becomes a rule in `@layer utilities` , plus the theme variables it uses, the Preflight reset (13.13) in `@layer base` , and needed `@property` registrations.

`code/08_tailwind.css` and part of `code/08_tailwind.html` :

```
@import "tailwindcss" source(none);    /* don't auto-scan the folder ...        */
@source "./08_tailwind.html";           /* ... scan exactly this file             */

@theme {                                /* tokens -> CSS variables AND utilities */
  --color-brand: #2563eb;               /* enables bg-brand, text-brand, ...     */
  --color-brand-dark: #1d4ed8;
}
```

```
<article class="w-[170px] rounded-xl border border-slate-300 bg-white p-3.5">
  <h3 class="text-base font-bold">Trail shoes</h3>
  <p class="mt-1 mb-3 text-slate-600">$89</p>
  <button class="w-full rounded-lg bg-brand p-2 text-white hover:bg-brand-dark">Add to cart</button>
</article>
```

(v4 auto-detects sources, skipping `.gitignore`d files; `source(none)` + `@source` just makes this example deterministic.) `code/08_tailwind_build.mjs` runs the real CLI and inspects the output:

```
classes used in the HTML: 28
utility rules generated:   28
output: 7623 bytes (6513 minified)
top-level at-rules: layer properties | layer theme, base, components, utilities | layer theme  | layer base  |
layer utilities  | property --tw-border-style | property --tw-leading ...
theme variables emitted: --color-amber-50 --color-amber-500 --color-amber-600 --color-amber-700 --color-slate-300
--color-slate-600 --color-slate-900 --color-white --color-brand --color-brand-dark

  .p-3\.5 {
    padding: calc(var(--spacing) * 3.5);
  }
  .bg-brand {
    background-color: var(--color-brand);
  }
  .w-\[170px\] {
    width: 170px;
  }
    .hover\:bg-brand-dark:hover {
      background-color: var(--color-brand-dark);
    }
```

- **Only what you use**: 28 classes → 28 rules, and only the 10 colour variables they reference (of hundreds). The whole stylesheet, reset included, is ~6.5 KB minified (whitespace stripped), before gzip/Brotli.
- **Layers**: `theme, base, components, utilities` — utilities are strongest, so a utility beats a component class without specificity games; your unlayered CSS beats all (13.4).
- **Spacing is a formula**: `p-3.5` = `calc(var(--spacing) * 3.5)`, `--spacing: 0.25rem` → 14px; change one variable to rescale the UI.
- **Arbitrary values** in brackets (`w-[170px]`) escape the scale.
- **Variants** (state/screen prefixes): `hover:` wraps the rule in `@media (hover: hover)` (in the full output) so touch devices avoid "sticky hover"; `md:` becomes `@media (width >= 48rem)`.

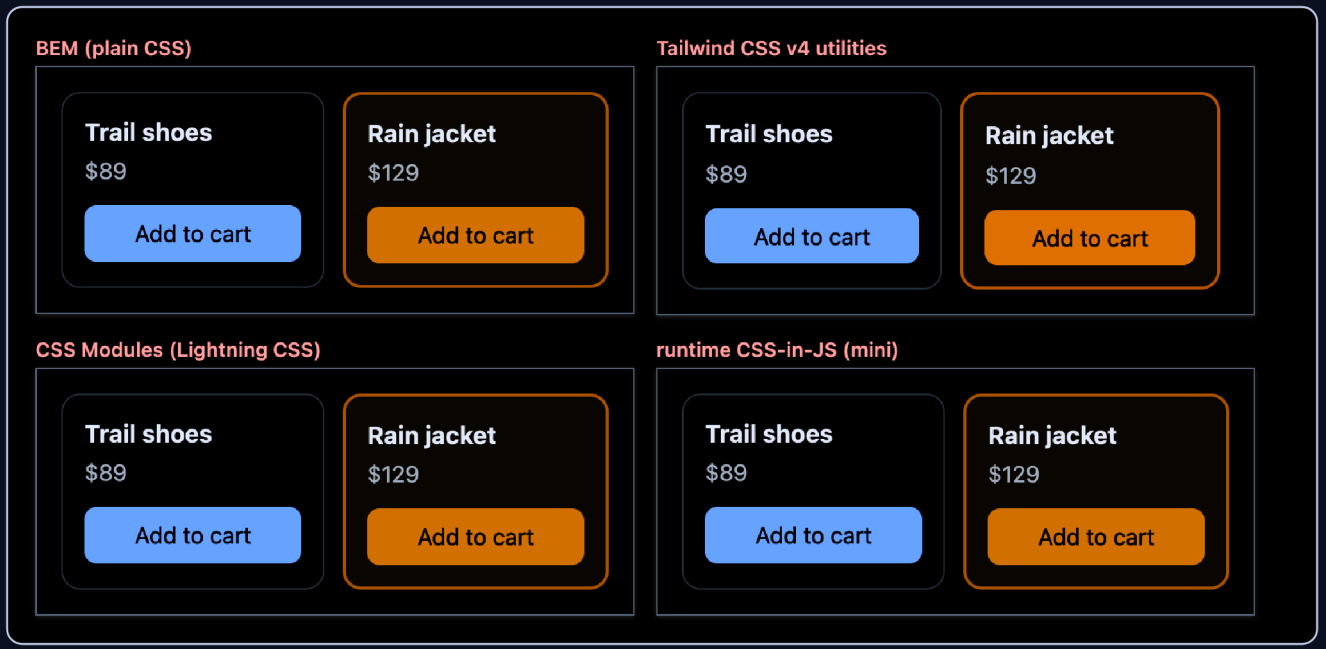


Figure 13.4 — The same two cards built four ways (13.7–13.10): identical pixels; different places styles live and how they're scoped.

Trade-offs

Pro	Con
Tiny, bounded CSS; no dead code	Long `class` attributes; HTML harder to scan
No naming, no specificity fights	Repetition unless you extract **components** (React/Vue components or template partials)
Design constraints built in	Learning the vocabulary; team must accept styling in markup
Easy to delete/refactor locally	Class names must be static strings (`bg-${color}-500` built at runtime won't be found)

Common mistakes

- **Dynamic class names** (`'text-' + color` , `'bg-' + c + '-500'`): never in the source in full, so no CSS. Map to full names: `{ red: 'text-red-600', blue: 'text-blue-600' }[color]` .
- Recreating BEM with `@apply` (copies utilities into your own rule) everywhere — losing both approaches' benefits.
- Forgetting focus styles: utilities don't add them (`focus-visible:ring-2` does).

In the interview

> **30-second answer:** "Utility-first CSS composes small single-purpose classes in the markup. Tailwind scans sources and generates only the utilities used, into cascade layers, so CSS stays bounded with no dead code or specificity war; v4 keeps configuration in CSS (`@theme`). The costs are verbose markup and a learning curve; I control repetition by extracting components, and class names must appear in full in the source."

13.9 CSS Modules

What it is

A **CSS Module** is a normal CSS file (`*.module.css` by convention) whose class names are **local by default**: the build swaps each class for a generated, collision-free name and gives JavaScript an object mapping your short names to the generated ones. You write `.title` ; the page gets `.title_14He7q` ; the component writes `className={styles.title}` . Built into Vite, Next.js, webpack's css-loader, Parcel and others.

How it works

1. Import from a component: `import styles from './Card.module.css'` .
2. At build time the compiler renames every class and `@keyframes` name by a pattern (file, class, a content hash).
3. It emits the renamed CSS into the bundle and returns the **name map** as the module's default export.
4. `composes: other` adds another class's generated name to the export, so the element gets both.
5. `:global(...)` opts a selector out of renaming.

`code/09_card.module.css` :

```
.card   { width: 170px; padding: 14px; border: 1px solid #cbd5e1; border-radius: 12px; background: #fff; }
.title  { margin: 0; font-size: 16px; }
.button { width: 100%; background: #2563eb; /* … */ }
.featured { composes: card; border: 2px solid #f59e0b; background: #fffbeb; }
.featured .button { background: #d97706; }
:global(body) { font: 15px/1.4 system-ui, sans-serif; margin: 16px; display: flex; gap: 12px; }
```

`code/09_css_modules.mjs` compiles it with Lightning CSS and renders the component (Figure 13.4, third panel):

```
import styles from "./card.module.css"  ->  styles = {
  button: 'button_14He7q',
  card: 'card_14He7q',
  featured: 'featured_14He7q card_14He7q',
  price: 'price_14He7q',
  title: 'title_14He7q'
}

Generated CSS (first rules):
.card_14He7q {
  background: #fff;
  border: 1px solid #cbd5e1;
  border-radius: 12px;
  width: 170px;
  padding: 14px;
}

.title_14He7q {
  margin: 0;
  font-size: 16px;
}

wrote 09_css_modules.out.css and 09_css_modules.html
```

`.title` in another module gets a different hash, so they never collide; `styles.featured` holds **two** names because of `composes`.

Trade-offs

Pro	Con
Real CSS (all features, DevTools, cached as a static file)	Needs a build tool and JS imports
Zero runtime cost	Styling by props means toggling classes (`clsx(styles.btn, big && styles.big)`)
Scoping enforced by the tool, not discipline	Generated names unreadable in production (fine in dev with a readable pattern)
Unused module files are detectable (no import → dead)	Unused classes *within* a file aren't removed

Don't confuse with "CSS module scripts": `import sheet from './x.css' with { type: 'css' }` returns a `CSSStyleSheet` (for shadow DOM) — a different, newer browser feature, **not Baseline** (no Safari as of September 2026).

13.10 CSS-in-JS

What it is

CSS-in-JS writes styles in JavaScript beside the component, often computed from props (component inputs):

- **Runtime** libraries (styled-components, Emotion): in the browser, while rendering, they **serialise** the style object, hash it into a class name and inject a rule into a `<style>` element.
- **Zero-runtime / compile-time** libraries (vanilla-extract, Linaria, Panda CSS, StyleX): authored in JS/TS, but extracted to static CSS at build time, like CSS Modules.

How runtime CSS-in-JS works — built in 25 lines

`code/10_mini_css_in_js.js` is a toy of the mechanism:

```
function css(styles) {
  const key = JSON.stringify(styles);
  if (cache.has(key)) return cache.get(key);          // same styles -> same class, inject once
  const cls = 'css-' + hash(key);                     // name derived from content
  sheet.insertRule(`.${cls}{${toCss(styles)}}`);      // + nested '&:hover' rules
  cache.set(key, cls);
  return cls;
}
// a component: styles depend on props
el.className = css({ border: featured ? '2px solid #f59e0b' : '1px solid #cbd5e1', /* … */ });
```

`code/10_css_in_js.mjs` renders two cards in jsdom (the fourth panel of Figure 13.4 is the same code in Chrome, `code/10_css_in_js.html`):

```
card 1 classes: css-1i6k3fq css-1qt5iag css-1d6s3yc css-11at6p7
card 2 classes: css-kegznd css-1qt5iag css-1d6s3yc css-n2x86t

rules injected at runtime (8):
  .css-1i6k3fq { width: 170px; padding: 14px; border-radius: 12px; border: 1px solid rgb(203, 213, 225); backgro
  .css-1qt5iag { margin: 0px; font-size: 16px; }
  ...
  .css-kegznd { width: 170px; padding: 14px; border-radius: 12px; border: 2px solid rgb(245, 158, 11); backgroun
  .css-n2x86t { width: 100%; padding: 8px; border: 0px; border-radius: 8px; background: rgb(217, 119, 6); color
  .css-n2x86t:hover { filter: brightness(0.9); }

progress bar rendered at 0..100%: 101 extra rules
with width: var(--pct) instead: 1 class (css-1gyb3r) + style="--pct: 42%"
```

- Identical styles share a class (`css-1qt5iag` for both titles) — the cache deduplicates.
- Different props → **different classes**.
- **The cost**: a continuously changing value (a progress bar) creates a rule per value — 101 rules — and serialising, hashing and inserting happen **during render, on the main thread** (and inserted rules trigger style recalculation), hurting responsiveness. The standard fix is the native one: one static rule with a custom property (`width: var(--pct)`) set inline.

Trade-offs

Pro	Con
Styles colocated with components; prop-driven styles are natural	Runtime cost: JS bundle + style work during render (hurts the Core Web Vitals INP — responsiveness — and LCP — main-content speed, §14.3)
Scoped automatically; dead styles die with the component	**Server rendering** (SSR — HTML built on the server) needs extra setup to collect styles; poor fit for **React Server Components** (server-only components that ship no JS; runtime libraries have needed *client components*)
TypeScript types for style props (zero-runtime libs)	CSS can't be cached separately from JS (runtime libs)

The ecosystem has moved to zero-runtime tools, CSS Modules and Tailwind. In March 2025 the styled-components maintainer announced **maintenance mode** (bug fixes only) and advised against it for new projects, citing its poor fit with React Server Components. It's still maintained — v6.4 (April 2026) added RSC-friendly theming via custom properties and faster re-renders — so existing projects are safe, but the advice for new work stands.

13.11 Choosing an approach

	BEM (plain CSS)	Utility-first (Tailwind)	CSS Modules	Runtime CSS-in-JS	Zero-runtime CSS-in-JS
Scoping	Convention	N/A (no custom selectors)	Build tool (renamed classes)	Generated classes	Build tool
Build step	Optional	Required	Required	No (runtime)	Required
Runtime cost	None	None	None	**Yes**	None
Dynamic styles	Classes / custom props	Classes / custom props	Classes / custom props	Props → styles	Variants / custom props
Dead-code removal	Manual	Automatic	Per file	Automatic	Automatic
CSS size growth	Grows with features	Plateaus	Grows with features	Grows with features	Grows / atomic (one class per declaration, StyleX) plateaus
Works without a JS framework	Yes	Yes	Needs a bundler	No	Needs a bundler
Good fit	Content sites, CMS themes, small teams	Product UIs, design systems with fixed scale	React/Vue apps wanting "just CSS"	Legacy apps already using it	TS-heavy apps wanting typed styles

Practical defaults in 2026:

1. **Content site / no framework** → plain CSS with layers, nesting, BEM-like names or `@scope`, and tokens.
2. **React/Vue/Svelte app** → Tailwind *or* CSS Modules (Svelte and Vue also scope `<style>` blocks natively) + tokens as custom properties.
3. **Always**: reset and tokens in low layers, dynamic values as custom properties, and don't mix three approaches in one codebase.

In the interview

30-second answer: "CSS's global namespace and specificity cause most maintenance pain. BEM solves it by convention with flat single-class names; CSS Modules by renaming classes at build time; utility-first by generating single-purpose classes so you rarely write new CSS; CSS-in-JS by generating classes from JS. I'd pick CSS Modules or Tailwind for a component app, plain CSS with layers and BEM or `@scope` for a content site, and avoid runtime CSS-in-JS in new projects for its render-time cost and poor fit with server components."

- *Migrating a big legacy stylesheet?* → Wrap it in a low `@layer legacy`, write new components in a higher layer with the chosen approach, delete legacy rules as pages migrate.

13.12 Preprocessors (Sass) and PostCSS

What it is

A **preprocessor** is a language that compiles to CSS. **Sass** (SCSS syntax, a superset of CSS) dominates; Less and Stylus are older. **PostCSS** instead parses CSS into a tree and runs plugins on it (Autoprefixer adds **vendor prefixes** like `-webkit-` that older browsers needed; others lower new syntax). **Lightning CSS** is a fast all-in-one compiler written in Rust (lowering, prefixing, minifying, CSS Modules) — used in 13.1 and 13.9.

What Sass adds

`code/11_sass_example.scss` — **(not run: Sass isn't in this book's toolchain)**:

```
@use "sass:color";
@use "sass:math";
$brand: #2563eb;                        // compile-time variable
$sizes: (sm: 12px, md: 16px, lg: 20px);

@mixin focus-ring($c: $brand) { outline: 2px solid $c; outline-offset: 2px; }
%button-base { border: 0; font: inherit; }   // placeholder for @extend

.card {
  padding: math.div(24px, 2);           // -> 12px
  &__title { font-size: 20px; }         // -> .card__title (string joining)
  &--featured { border-color: color.adjust($brand, $lightness: -10%); }
}
.button {
  @extend %button-base;
  &:focus-visible { @include focus-ring; }
}
@each $name, $size in $sizes { .text-#{$name} { font-size: $size; } }   // generates 3 classes
```

Modern Sass: use the module system (`@use` / `@forward` — each file loaded once, names private unless shared); `@import` is **deprecated** since Dart Sass 1.80 (October 2024) and will be removed in Dart Sass 3.0; global functions like `darken()` are deprecated for `color.adjust()` / `color.scale()`; **Dart Sass** is the only maintained implementation (LibSass and node-sass are deprecated). A Sass `$brand` can't change per section at runtime — it's replaced at compile time.

What native CSS now covers

`code/11_native.html` rewrites the Sass ideas in plain CSS (`code/11_native.mjs` checks Chrome; `oklch(L C H)` = lightness, chroma, hue in a perceptually even space):

```
:root { --brand: #2563eb; --radius: 8px; }
.card {
  padding: calc(24px / 2);
  .card-title { font-size: 20px; }
  &.is-featured { border-color: oklch(from var(--brand) calc(l - 0.1) c h); }  /* relative colour */
}
.button {
  background: var(--brand);
  &:hover { background: color-mix(in oklch, var(--brand), black 15%); }
}
.theme-green { --brand: #16a34a; }     /* runtime re-theming */
```

```
padding: calc(24px / 2)                 -> 12px
border-color: oklch(from --brand l-0.1) -> oklch(0.44613 0.215194 262.885)
.button background (var(--brand))        -> rgb(37, 99, 235)
same rule under .theme-green             -> rgb(22, 163, 74)
hover: color-mix(in oklch, brand, black 15%) -> oklch(0.464211 0.182915 262.885)
```

Sass feature	Native CSS today	Status (Sept 2026)
`$variables`	Custom properties `--x` — **live**, inherit, change per element and at runtime (the green button)	Widely available
Nesting	Native nesting (13.1) — no `&__` joining	Widely available
`math.div`, arithmetic	`calc()`, `min()`, `max()`, `clamp()`, `round()`, `mod()`	`calc` etc. widely; `round()/mod()/rem()` newly available (2024)
`darken()` / `color.adjust`	`color-mix()` (widely) and relative colour syntax `oklch(from …)` (newly available, Sept 2024)	✓
Partials / `@use`	`@import` (blocks parallel loading — avoid in production) or bundler concatenation; `@import … layer()`	✓
`@mixin`	CSS `@mixin` / `@apply` spec still being designed	No browser
`@function`	Custom functions `@function --name()`	**Chromium only** (139+)
`@if` / `if()`	`if()` function	**Chromium only** (137+)
`@each` / loops generating classes	None — generate at build time (or use utilities)	—
`@extend`	None (rarely a good idea: it rewrites selectors unpredictably)	—

Verdict. For new projects native CSS covers most daily Sass use; keep Sass for mixins, loops or an existing Sass design system. A PostCSS/Lightning CSS step for prefixes and lowering is still common either way.

13.13 Resets and normalize

What it is

Every browser applies a **user-agent stylesheet**: 8px body margin, heading and paragraph margins, bullets and 40px list padding, form controls with their own small font. A **reset** removes or neutralises these. Three generations:

1. **Hard resets** (Eric Meyer, 2007/2011): zero almost everything, including list styles and heading sizes — you restyle it all.
2. **normalize.css** (2012): keep useful defaults but make browsers **consistent** and fix bugs.
3. **Modern minimal resets** (Andy Bell, Josh Comeau, Tailwind's Preflight): a dozen rules fixing today's real annoyances; browsers are consistent enough that normalizing matters less.

A modern reset, line by line

`code/12_reset.css` — in the lowest layer so anything can override it without specificity fights:

```css
@layer reset {
  *, *::before, *::after { box-sizing: border-box; }          /* width includes padding + border */
  * { margin: 0; }                                             /* add spacing on purpose */
  html { -webkit-text-size-adjust: none; text-size-adjust: none; }  /* no iOS text inflation */
  body { min-height: 100dvh; line-height: 1.5; -webkit-font-smoothing: antialiased; }
  img, picture, video, canvas, svg { display: block; max-width: 100%; }
  input, button, textarea, select { font: inherit; }          /* controls don't inherit fonts by default */
  p, h1, h2, h3, h4, h5, h6 { overflow-wrap: break-word; }    /* long URLs can't overflow */
  h1, h2, h3, h4, h5, h6 { text-wrap: balance; }
  p { text-wrap: pretty; }
  :where(ul, ol)[role="list"] { list-style: none; padding: 0; }
  @media (prefers-reduced-motion: reduce) {
    *, *::before, *::after { animation-duration: 0.01ms !important; transition-duration: 0.01ms !important;
scroll-behavior: auto !important; }
  }
}
```

code/12_reset.mjs loads code/12_reset.html at 400px, measures, adds the reset, measures again:

```
property                              browser default          with reset
body margin                           8px                      0px
h1 margin-block                       21.44px 21.44px          0px 0px
h1 font-size                          32px                     32px
#box width (200px + padding + border) 250px                    200px
img display / rendered width          inline / 800px           block / 400px
button font                           13.3333px Arial          16px Times
ul list-style / padding-left          disc / 40px              none / 0px
p scrollWidth (overflow?)             606px of 384px           400px of 400px
```

Margins gone; `box-sizing` makes `width: 200px` really 200px (§7.2); the 800px image no longer overflows 400px; the button uses the page font (Times here since the demo sets none — normally your `body` font); the `role="list"` list lost its bullets; the long URL wraps (606px in a 384px box before). Heading sizes untouched — this reset only removes *surprises*.

- **`role="list"`**: Safari's VoiceOver stops announcing a list when `list-style: none` is applied; `role="list"` restores it, so the reset strips bullets only from lists carrying it.
- **`:where()`** keeps that rule low-specificity (13.2); the **`layer`** keeps *all* of it below your styles (13.4).
- **`text-wrap: pretty`** is not Baseline (no Firefox, September 2026) — harmless where unsupported. `text-size-adjust` is non-standard/partial, hence both spellings.
- **Never** put `outline: none` / `*:focus { outline: 0 }` in a reset — it removes the keyboard focus indicator (§3.7).

13.14 Theming and dark mode with custom properties

What it is

A **theme** is a set of values (colours, radii, fonts, spacing) applied consistently. Build it with **design tokens** as **custom properties** in three tiers, plus `color-scheme` (which light/dark modes the page supports) and `light-dark()` (picks one of two values by active mode). §10.5 introduced `prefers-color-scheme` and basic dark mode; §6.9 custom properties. This section combines them into an architecture.

The three tiers of tokens

Tier	Example	Who uses it
Primitive (reference) tokens	`--blue-600: #2563eb`	Only the semantic tokens
Semantic (system) tokens	`--color-accent`, `--color-surface`, `--color-text`	Components (a button references `--color-accent`)
Component tokens (optional)	`--card-bg: var(--color-surface)`	One component; lets one instance be tweaked

Components never mention a raw colour; dark mode or a new brand changes **only semantic tokens**.

How it works

1. `color-scheme: light dark` on `:root` declares support; the browser themes its own UI (controls, scrollbars, default background), and **`light-dark(a, b)` picks `a` or `b` by the element's used colour scheme** (so `light-dark(#fff, #000)` under a dark OS gives `#000`). Baseline newly available since May 2024 (colours only; image values newer still).
2. Each semantic token gets a `light-dark()` pair — no duplicated `@media` block.
3. A user choice **overrides** the OS by pinning `color-scheme`: `:root[data-theme="dark"] { color-scheme: dark }`. Because `light-dark()` follows `color-scheme`, every token flips without duplication.
4. A tiny **blocking script in `<head>`** reads the saved choice from `localStorage` (per-site key–value storage surviving reloads) and sets `data-theme` *before first paint*, so there's no flash of the wrong theme (a justified exception to "don't block rendering", §14.2). It goes on `<html>` because tokens live on `:root` and `<body>` doesn't exist yet.
5. Other axes (brand, density, high contrast) are attribute-scoped token overrides: `[data-brand="green"] { --color-accent: … }`. Custom properties inherit, so an override on any element re-themes its subtree.

`code/13_theming.html` (key parts):

```
<meta name="color-scheme" content="light dark">
<script>
  try { const t = localStorage.getItem('theme');
        if (t === 'light' || t === 'dark') document.documentElement.dataset.theme = t; } catch {}
</script>
```

```
@layer tokens, base, components;
@layer tokens {
  :root {
    --blue-600: #2563eb;  --blue-300: #93c5fd;  --slate-900: #0f172a; /* … primitives */
    color-scheme: light dark;
    --color-bg:      light-dark(var(--slate-50), var(--slate-900));
    --color-surface: light-dark(#fff, var(--slate-800));
    --color-text:    light-dark(var(--slate-900), var(--slate-200));
    --color-accent:  light-dark(var(--blue-600), var(--blue-300));
  }
  :root[data-theme="light"] { color-scheme: light; }
  :root[data-theme="dark"]  { color-scheme: dark; }
  [data-brand="green"] { --color-accent: light-dark(#15803d, #86efac); }
}
@layer components {
  .card a { color: var(--color-accent); }
  .btn { background: var(--color-accent); color: var(--color-surface); }
}
```

```
select.addEventListener('change', () => {
  if (select.value === 'system') delete root.dataset.theme; else root.dataset.theme = select.value;
  try { select.value === 'system' ? localStorage.removeItem('theme') : localStorage.setItem('theme',
select.value); } catch {}
});
```

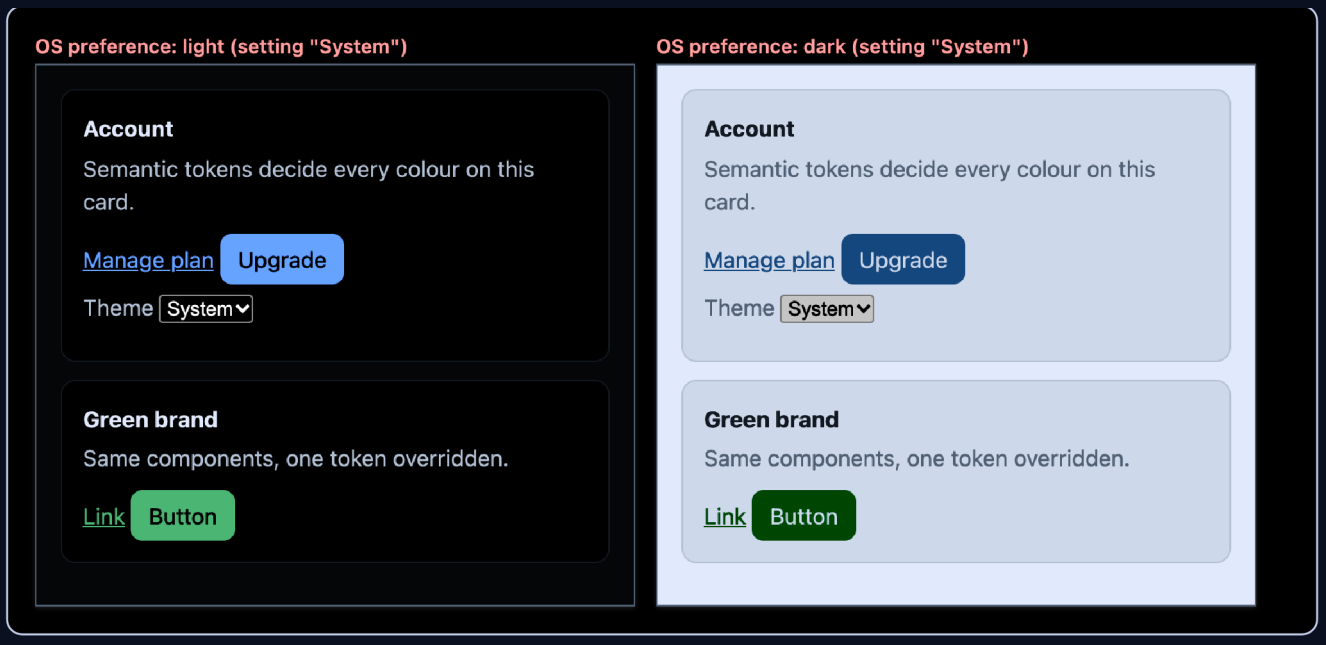


Figure 13.5 — Setting "System" under emulated light and dark OS preferences; the select box and page background are themed by `color-scheme` with no CSS of their own.

`code/13_theming.mjs` tests every OS preference × page setting, then reloads to test persistence:

```
OS pref  setting   page bg              text                accent (link)       green-brand link
light    system    rgb(248, 250, 252)   rgb(15, 23, 42)     rgb(37, 99, 235)    rgb(21, 128, 61)    saved=null
light    light     rgb(248, 250, 252)   rgb(15, 23, 42)     rgb(37, 99, 235)    rgb(21, 128, 61)    saved=light
light    dark      rgb(15, 23, 42)      rgb(226, 232, 240)  rgb(147, 197, 253)  rgb(134, 239, 172)  saved=dark
dark     system    rgb(15, 23, 42)      rgb(226, 232, 240)  rgb(147, 197, 253)  rgb(134, 239, 172)  saved=null
dark     light     rgb(248, 250, 252)   rgb(15, 23, 42)     rgb(37, 99, 235)    rgb(21, 128, 61)    saved=light
dark     dark      rgb(15, 23, 42)      rgb(226, 232, 240)  rgb(147, 197, 253)  rgb(134, 239, 172)  saved=dark

after reload: data-theme=dark  body bg=rgb(15, 23, 42)  <select> shows "dark"
```

"System" follows the OS, an explicit choice beats it, the brand override keeps its own light/dark pair, and after reload the head script applied "dark" before rendering.

Going further

- **Typed tokens with `@property`** (Baseline newly available since July 2024): `@property --accent { syntax: '<color>'; inherits: true; initial-value: #2563eb; }` lets the browser *interpolate* the token (animated theme changes) and rejects invalid values.
- **Fallback without `light-dark()`**: light tokens on `:root`, overridden in `@media (prefers-color-scheme: dark)` and `[data-theme=dark]` — the Chapter 10 pattern.
- **Contrast**: check every token pair (text on bg/surface, accent on surface) at ≥ 4.5:1 in *both* themes (§11.4).
- **`contrast-color()`** (returns black or white for readable text on a background) became Baseline in April 2026 — useful for user-chosen brand colours.

Common mistakes

- `light-dark()` without `color-scheme` → always the light value.
- Components using primitive tokens (`var(--blue-600)`) — dark mode then needs component changes.
- Setting the theme in a `useEffect` (React hook running after render) or `DOMContentLoaded` handler → paints light, then flashes dark.
- Storing the theme only in a cookie read by JS after load — same flash (unless rendered server-side from the cookie).

In the interview

> **30-second answer:** "I define primitive tokens, then semantic tokens like `--color-surface` that components use. For dark mode I set `color-scheme: light dark` and give each semantic token a `light-dark()` pair; a user override pins `color-scheme` on the root via `data-theme`, saved in localStorage and applied by a tiny inline head script to avoid a flash. Extra themes are scoped token overrides, which work because custom properties inherit."

Chapter 13 TL;DR

- **Nesting** (Baseline 2023): nested selectors are relative to the parent; `&` = parent, like `:is()` (highest specificity of the parent list); no `&__` string joining (Chrome dropped that rule); nested `@media` works; Lightning CSS can lower it.
- **`:is()` / `:where()`**: "any of"; forgiving lists; `:is()` = max argument specificity, `:where()` = 0 → `:where()` for resets and library defaults.
- **`:has()`** (Baseline 2023): parent / previous-sibling / state selector (`.card:has(img)`, `h3:has(+ .subtitle)`, `label:has(:checked)`, `html:has(dialog[open])`); no nested `:has()`; anchor it.
- **`@layer`**: declare order first; later layers win regardless of specificity (a class in `components` beat `#app .btn.btn-primary` in `vendor`); unlayered beats layers; `!important` flips; `revert-layer`; `@import … layer()`.
- **`@scope`** (Baseline March 2026): `@scope (.card) to (.content)` donut scope; `:scope`; proximity breaks specificity ties; prelude-less `@scope` in inline `<style>`.
- **Logical properties**: `inline` / `block` + `start` / `end`; one stylesheet for LTR, RTL and vertical text.
- **Architecture**: global namespace + specificity + dead code are the problems. **BEM** (`block__element--modifier`), **Tailwind v4** (generated utilities in `@layer utilities`, only what's used — 28 classes → 28 rules, ~6.5 KB), **CSS Modules** (`.title` → `.title_14He7q`, `composes`), **CSS-in-JS** (runtime hash + inject during render, 101 rules for 101 progress values; prefer zero-runtime or custom properties).
- **Sass**: variables, mixins, loops, `&__` joining; `@use` not `@import`; native CSS covers variables, nesting, colour maths — not mixins/loops (`@function` / `if()` Chromium-only).
- **Reset**: small modern reset in `@layer reset`; never remove focus outlines.
- **Theming**: primitive → semantic → component tokens; `color-scheme` + `light-dark()` (Baseline 2024); `[data-theme]` pins `color-scheme`; persist in localStorage, apply in `<head>` before first paint.

Next: Chapter 14 — Performance, SEO, Interview Prep and Cheat Sheets.